

ECTE331
Project
Sanjit Mathur
8303551
[GitHub](#)

Part 1
Fault Tolerant Drone Navigation System

1.1 Introduction and Overview

The Triple Modular Redundancy (TMR) principle, which is frequently employed in safety-critical embedded systems (aerospace, medical devices, nuclear control) where hardware or software faults must be concealed without disrupting operation, is used in this component to implement a fault-tolerant altitude monitoring system for an autonomous drone.

Three separate altitude sensors (A, B, and C) are polled during each of the 20 control cycles that the system replicates. A majority-voting algorithm receives their readings, chooses the agreed-upon value, and eliminates outliers. The system switches to SAFE MODE, which stops regular functioning to safeguard the vehicle, if it is unable to reach an agreement for two consecutive cycles.

1.2 System Architecture

Class	Role
DroneAltitudeSensor	Models a single sensor with probabilistic fault injection
TMRController	Implements majority voting and reliability tracking
SensorReadException	Thrown when a sensor completely fails to produce a reading
DroneNavigationSystem	Main entry point; orchestrates sensor polling, logging, and cycle control
SystemReliabilityException	Thrown when two consecutive voting failures are detected

1.3 Sensor Fault Model

DroneAltitudeSensor is called every time. After generating a random integer in $[0, 100]$, readSensor() classifies the result into one of three scenarios:

Sensor Failure (15% probability): A SensorReadException is thrown by the method. This mimics a hardware dropout, in which the sensor is physically unable to generate a reading (e.g., bus timeout, power outage).

Corrupted Reading (15% probability): The sensor provides a result that is not within the acceptable range $[0, 200]$, either a negative value or a huge positive value (over 200 m). This mimics ADC overflow or electromagnetic interference.

Valid Reading (70% probability): Clamped to $[0, 200]$, the sensor provides the baseline altitude (100 m) plus a tiny random offset in $[0, 19]$.

This results in a total fault injection rate of 30%, which is purposefully high to put the TMR logic to the test.

1.4 TMR Voting Algorithm

The TMRController.applyVoting() method implements the following logic:

1. Collect valid readings: Iterate over all three sensor outputs. A reading is valid only if it is non-null and falls within [0, 200]. Failed (null) or corrupted (out-of-range) readings are moved to an outlier list and logged.
2. Minimum quorum check: At least two valid readings are required to attempt voting. If fewer than two are available, the failure handler is invoked.
3. Pairwise majority search: All pairs of valid readings are compared. If any two sensors agree on the same integer value, that value is adopted as the final altitude. This is the TMR majority decision.
4. Failure handling (handleFailure):
 - Increments a consecutive failure counter.
 - On the first failure, the system falls back to the previous known-good altitude and continues (degraded mode).
 - On the second consecutive failure, SystemReliabilityException is thrown, triggering SAFE MODE.
5. Reliability recovery: A successful majority vote resets the consecutive failure counter to zero and restores the status to "OK".

1.5 Logging

Every significant event is written to a timestamped log file (log_<timestamp>.txt) using BufferedWriter. Logged events include:

- Corrupted readings (with sensor ID and raw value)
- Sensor failures (exception message)
- Outlier detections
- Majority decisions (agreed sensors and altitude)

- Reliability failures and fallback decisions
- SAFE MODE activation
- System start and termination

This provides a complete, auditable trace of system behaviour essential for post-incident analysis in safety-critical systems.

PART 2 — Real-Time Robotic Arm Controller

2.1 Introduction

This component models a multi-threaded robotic arm control system with three concurrent tasks, each mapped to a different priority level:

Thread	Priority	Role
ArmLogger	LOW (P=1)	Writes diagnostic data to persistent storage
MotionPlanner	MEDIUM (P=5)	Computes joint trajectories for arm movement
SafetyMonitor	HIGH(P=10)	Monitors for unsafe joint positions; must respond promptly

All three threads contend for a shared resource (the MotorController lock), which creates the conditions for priority inversion a well-known real-time scheduling hazard documented in the Mars Pathfinder mission (1997).

Tasks 1–6 progressively build, diagnose, and mitigate this problem.

Task 1 – Basic Implementation

Task 1 establishes the baseline system: three independent threads, each printing their start and completion times. No shared resources or

synchronisation primitives are used. This confirms the threads run concurrently and that Java's priority scheduler gives higher-priority threads more CPU time. It also demonstrates that without locking, the safety monitor always completes promptly.

Task 2 - Synchronisation

Task 2 introduces a shared MotorController object protected by a synchronized block. All three threads must acquire the intrinsic lock before accessing the motor. This is the simplest form of mutual exclusion in Java.

The key observation is that whichever thread acquires the lock first holds it for its full work duration the others block.

Java's synchronized keyword provides mutual exclusion and visibility guarantees through the happens-before relationship, but offers no priority-aware scheduling.

Task 3 - Priority Inversion Demonstration

This is the core demonstration of the priority inversion problem. The scenario proceeds as follows:

1. ArmLogger (LOW, P=1) acquires the motor lock and holds it for 1500 ms.
2. SafetyMonitor (HIGH, P=10) starts 200 ms later and attempts to acquire the same lock — it blocks, despite having the highest priority.
3. MotionPlanner (MEDIUM, P=5) starts 400 ms later. Because it does NOT need the lock, it runs freely on the CPU, potentially preempting the low-priority ArmLogger.
4. This means ArmLogger is delayed from releasing the lock, which in turn delays SafetyMonitor — the highest-priority, most time-critical task in the system.

Priority inversion is the situation where a high-priority task's execution is effectively blocked by a medium-priority task through the indirect

mechanism of lock holding. The measured wait time for SafetyMonitor is substantially greater than the intrinsic lock-holding time of ArmLogger.

In safety-critical systems, this is catastrophic: the safety monitor cannot respond to hazardous arm positions in time.

Task 4 – Priority Inheritance Protocol

Task 4 introduces a custom PriorityInheritanceLock built on `java.util.concurrent.locks.ReentrantLock`, since Java provides no native priority inheritance.

Mechanism:

- When `lock()` is called, the implementation checks whether the current owner's priority is lower than the caller's.
- If so, it boosts the owner's priority to match the waiting high-priority thread using `Thread.setPriority()`.
- When `unlock()` is called, the owner's priority is restored to its original value.

Effect in the scenario:

- ArmLogger (P=1) acquires the lock.
- SafetyMonitor (P=10) attempts to acquire -> ArmLogger's priority is immediately boosted to P=10.
- MotionPlanner (P=5) can no longer preempt ArmLogger (now P=10), so ArmLogger finishes faster.
- SafetyMonitor acquires the lock sooner its measured wait time is significantly reduced compared to Task 3.

The volatile keyword on the owner field ensures that the priority-boosting thread sees the most recent owner reference without caching stale values across threads.

Task 5 - Priority Ceiling Protocol

Task 5 implements an alternative: the Priority Ceiling Protocol (PCP). A PriorityCeilingLock is initialised with a ceiling priority equal to Thread.MAX_PRIORITY (10).

Mechanism:

- The moment any thread acquires the lock, its priority is immediately raised to the ceiling regardless of which threads are waiting.
- Upon release, the original priority is restored.

Key difference from Priority Inheritance:

- Priority Inheritance is reactive it boosts only when a high-priority thread actually blocks.
- Priority Ceiling is proactive it boosts immediately at lock acquisition, preventing any medium-priority thread from ever preempting a lock holder.

Additional benefit: PCP also prevents deadlock in systems with multiple locks. If the ceiling is set to the maximum priority of any task that will ever use the lock, no lower-priority thread can acquire a second lock while holding the first, eliminating circular wait.

In the robotic arm scenario, ArmLogger's priority is raised to $P=10$ the instant it acquires the ceiling lock, so MotionPlanner ($P=5$) cannot run at all during the critical section. SafetyMonitor's wait time is minimised to approximately the lock hold time only.

Task 6 – Performance Evaluation

Task 6 repeats the three-thread scenario (baseline, inheritance, ceiling) for 5 runs each and computes the average SafetyMonitor wait time for comparison.

Expected results:

- Baseline: SafetyMonitor waits the full 1500 ms (approx.) that ArmLogger holds the lock, plus any additional delay from MotionPlanner preempting ArmLogger.

- Priority Inheritance: Wait time is significantly reduced, as ArmLogger finishes faster once its priority is boosted.
- Priority Ceiling: Wait time is minimised, as preemption is prevented from the very start of the critical section.

PART B — Thread Synchronisation and Communication

B.1 Introduction

This component demonstrates semaphore-based inter-thread dependency enforcement a classic concurrency problem where two threads must execute a set of functions in a specific partial order that cannot be achieved by simple sequential execution of either thread alone.

B.2 Dependency Graph

Six functions are split between two threads:

ThreadA: FuncA1 -> FuncA2 -> FuncA3

ThreadB: FuncB1 -> FuncB2 -> FuncB3

With the following cross thread dependencies:

FuncA1 must complete before FuncB1 can start

FuncB2 must complete before FuncA2 can start

FuncA2 must complete before FuncB3 can start

FuncB3 must complete before FuncA3 can start

B.3 Semaphore Design

A semaphore initialised to 0 acts as a signal: acquire() blocks the caller until another thread calls release(). This is conceptually equivalent to a binary event flag.

The use of `java.util.concurrent.Semaphore` rather than manual `wait()/notify()` pairs ensures:

- Atomicity: `acquire()` and `release()` are atomic operations.
- No missed signals: A `release()` before `acquire()` increments the permit count; the next `acquire()` succeeds immediately. There is no race condition between the signal and the wait.
- Determinism: The execution order is enforced regardless of OS scheduling decisions or thread start-up timing.

Shared result variables (`A1`, `A2`, etc.) are declared `volatile` in `SharedVariables`, which guarantees that after a `semaphore.release()`, the consuming thread's subsequent `semaphore.acquire()` will see the most recently written values, due to the happens-before guarantee through `java.util.concurrent` semaphore operations.

B.4 Verification (1000 Iteration)

`ThreadSyncApp` first performs a single demonstration run, printing all computed values and verifying them against the expected constants. It then repeats the experiment 1000 times in a loop, checking correctness after each run.

Result: All 1000 runs pass (0 failures), demonstrating that the semaphore-based synchronisation is deterministic, it produces the correct result regardless of thread scheduling variations. If any run produced a wrong value, it would indicate a race condition in the dependency enforcement.

This high-iteration verification is a standard technique in concurrent software testing to expose race conditions that are unlikely to manifest in a single run due to favourable scheduling.

